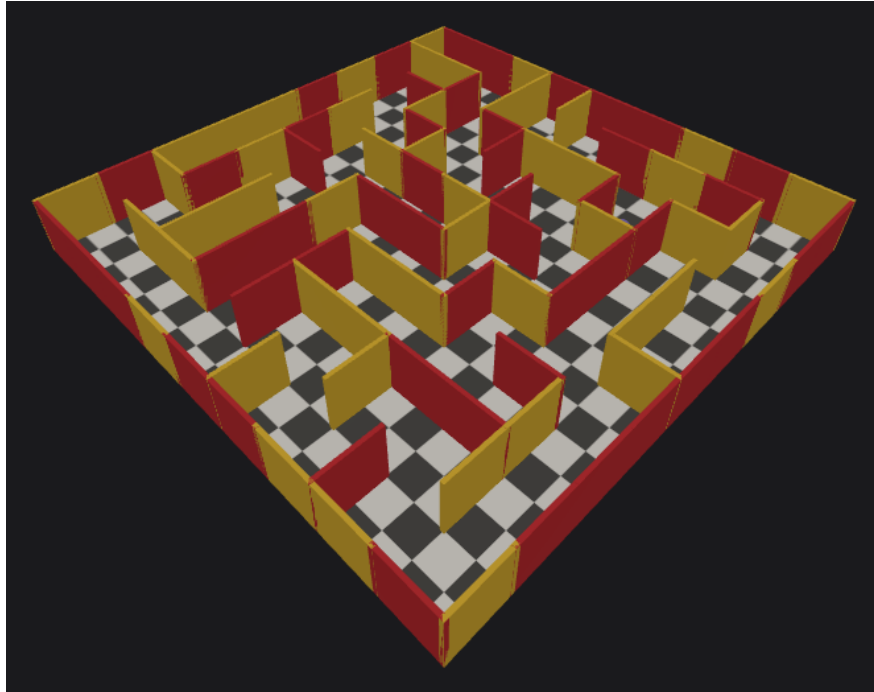


# Level 3: The Maze

How it looks, how it's generated, and how it plays — a walkthrough for the team

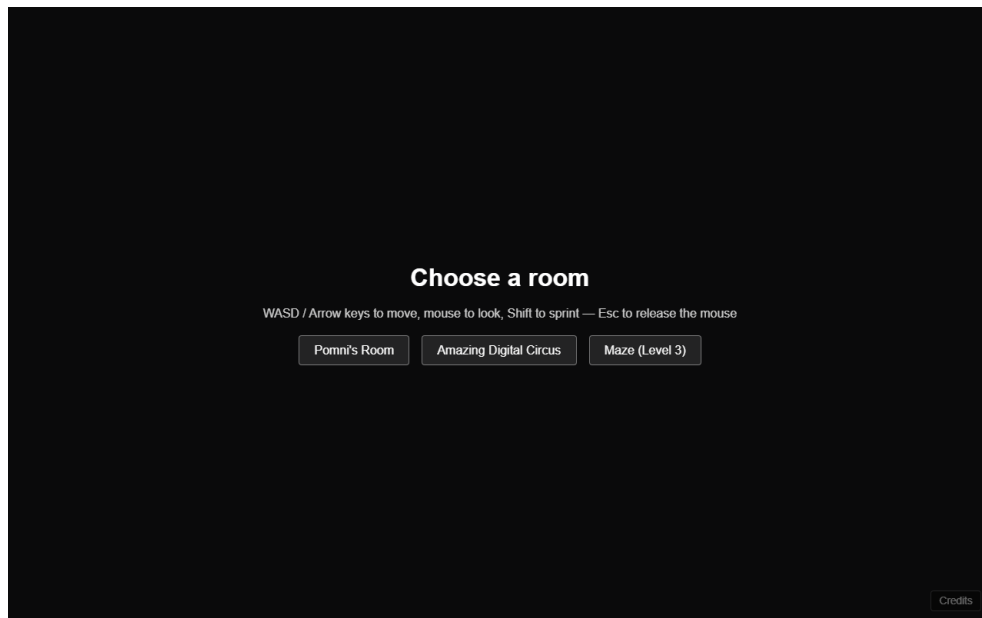


A freshly generated maze, viewed from above at an angle. Every wall segment doubles as a collision box, so what you see is exactly what the player can and can't walk through.

## What this level is

This is the Level 3 maze from the design doc: "Maze to find Jax then the exit." The player is dropped into a corner of the maze and has two objectives — find Jax somewhere inside it, then find the exit. Reaching the exit ends the level, whether or not Jax was found first, which leaves room for a later good/bad-ending split similar to the Ribbit and Kaufmo levels.

The maze is **generated in code** rather than modeled by hand in Blender (unlike the other two rooms in the game, which load pre-built .glb models). That was a deliberate choice: a generated maze produces its own exact collision data for free, so there is no risk of the visuals and the walkable area drifting apart. It also means the layout, size, and difficulty can all be tuned by changing a few numbers instead of re-modeling geometry.



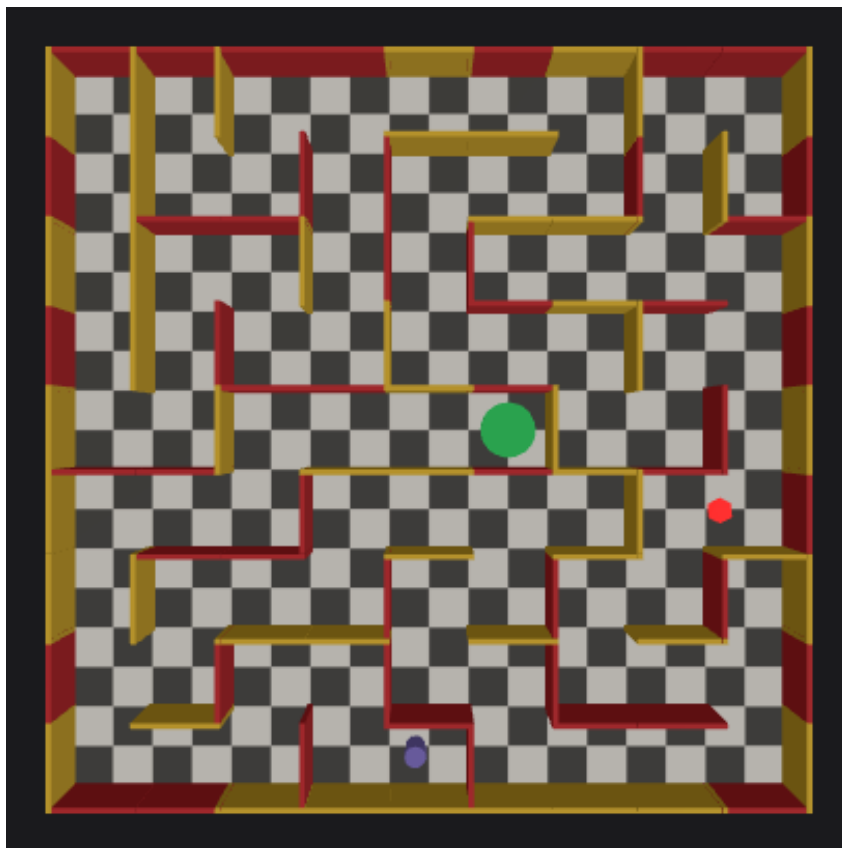
The room picker now includes "Maze (Level 3)" alongside the two existing rooms.

## How the layout is generated

The maze is a 9×9 grid of cells, each 3 units square. It's carved using a classic **recursive-backtracker** algorithm:

1. Start at one cell and mark it visited.
2. Look at its unvisited neighbors. If there are any, pick one at random, knock down the wall between the current cell and that neighbor, and move into it.
3. If there are no unvisited neighbors, back up to the last cell that still has one, and continue from there.
4. Repeat until every cell has been visited.

This guarantees the result is a "perfect" maze: every cell is reachable from every other cell, and there is exactly one path between any two points — no isolated pockets, no loops. That property was checked automatically (fully-connected across 20 random generations) before this was wired into the game.



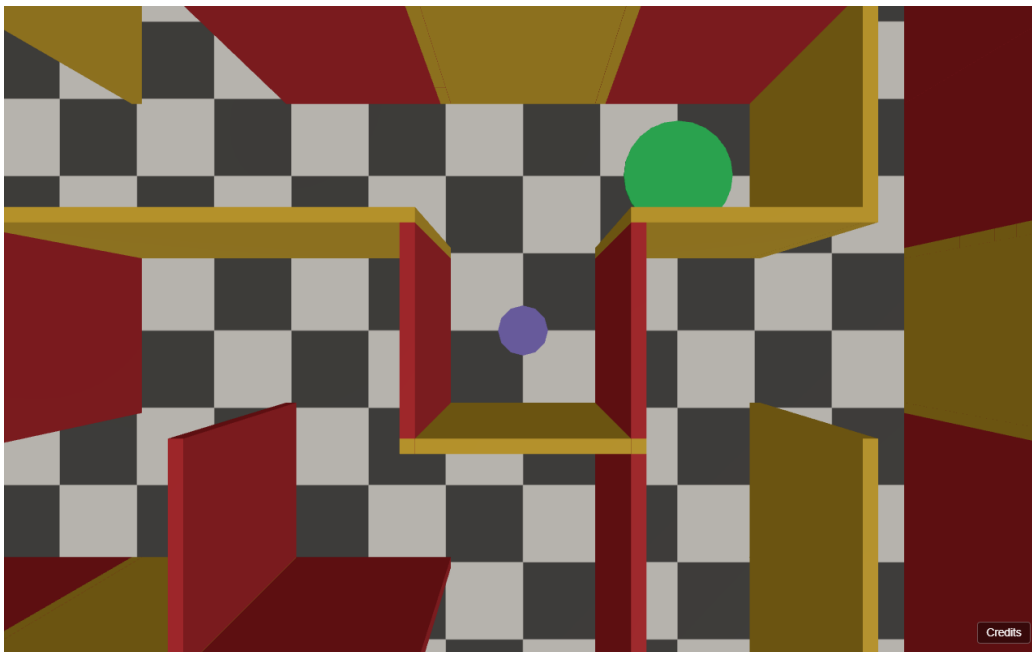
A generated layout from directly above. Green is the exit, the small purple dot is Jax, and the red dot is the patrolling hazard covered on the next page. The player always spawns in the bottom-left corner.

## Placing Jax and the exit

Once the layout is carved, the same generation pass runs a breadth-first search from the spawn cell to find the path distance to every other cell. That distance is used to place the two objectives so the level actually requires exploring the maze rather than being solvable by luck:

- **Exit** — placed at whichever cell is farthest (by path length) from spawn.
- **Jax** — placed at the farthest dead-end that isn't the exit, so the two objectives sit in different branches instead of overlapping or being right next to each other.

Jax also isn't standing still — he wanders a short back-and-forth patrol between his dead end and its one connecting corridor, so the trigger that reveals him covers that whole short stretch rather than one fixed point.

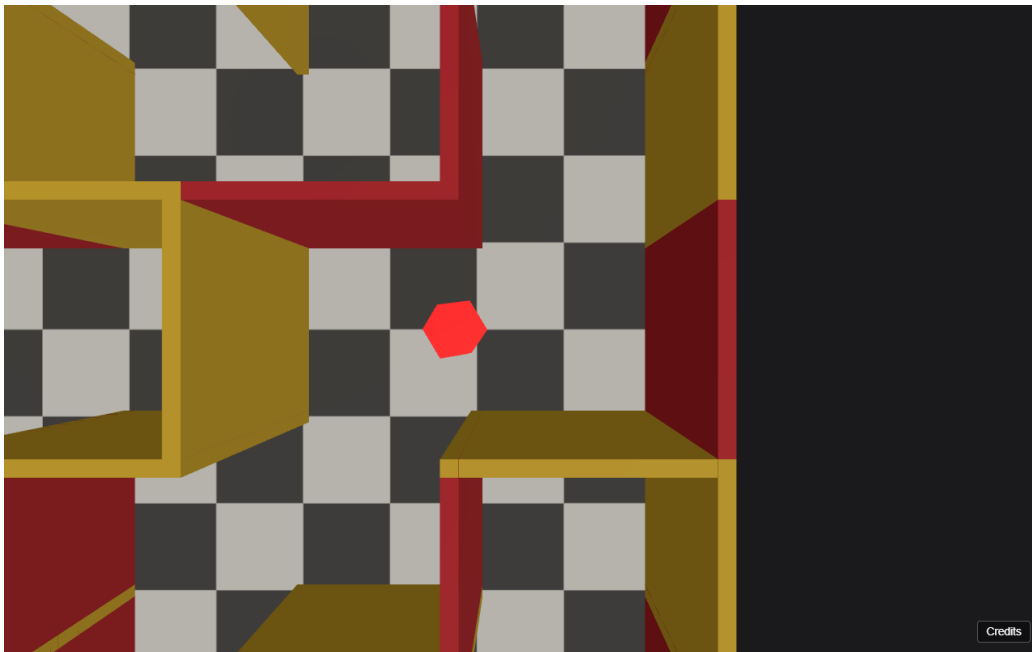


Both markers are currently simple placeholder shapes — a purple cylinder for Jax, a green disc for the exit — standing in for the real character model and exit dressing the team will add later. What matters underneath is the invisible trigger volume around each one: walking into it fires an event.

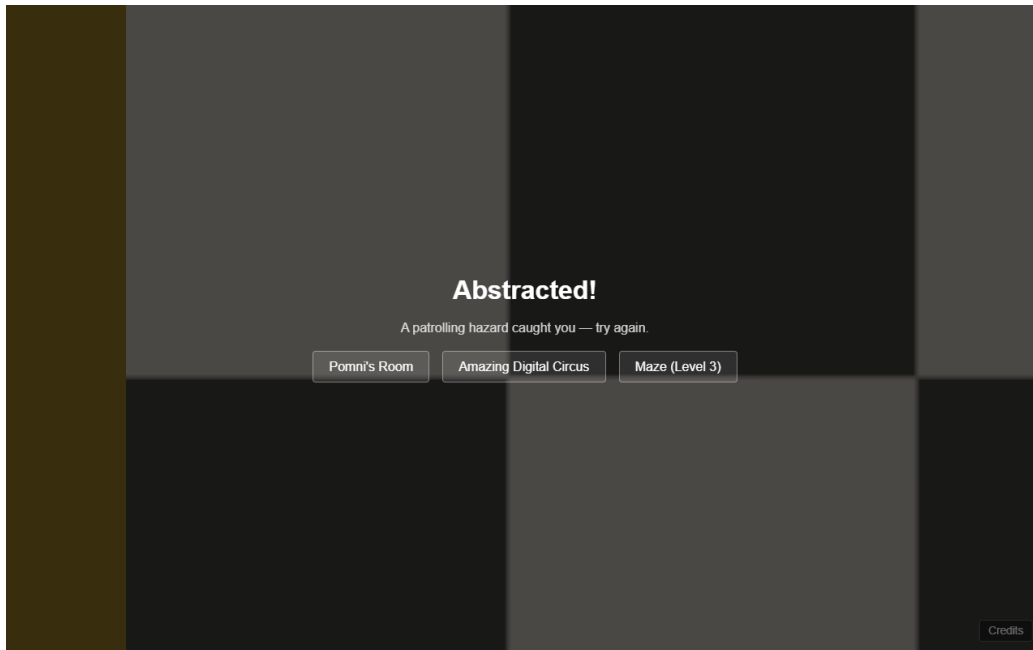
## Avoiding the hazard

The design brief for this level calls for "navigation + chase — find Jax, then find the exit together while avoiding abstracted monsters." To back that up, a patrolling hazard walks a stretch of cells taken directly from the shortest start-to-exit route (reconstructed via the same breadth-first search, this time following parent links back from the exit). That means the hazard sits somewhere along the path the player actually needs to take, not off in a side branch where it could be ignored.

Touching it ends the level immediately with an "Abstracted!" message and a retry, using the same trigger pattern already built for Jax and the exit — just checked against the hazard's current position every frame instead of a fixed volume, since it keeps moving.



The hazard, viewed from above — another placeholder shape (a spinning red icosahedron) standing in for a real abstracted-monster model.

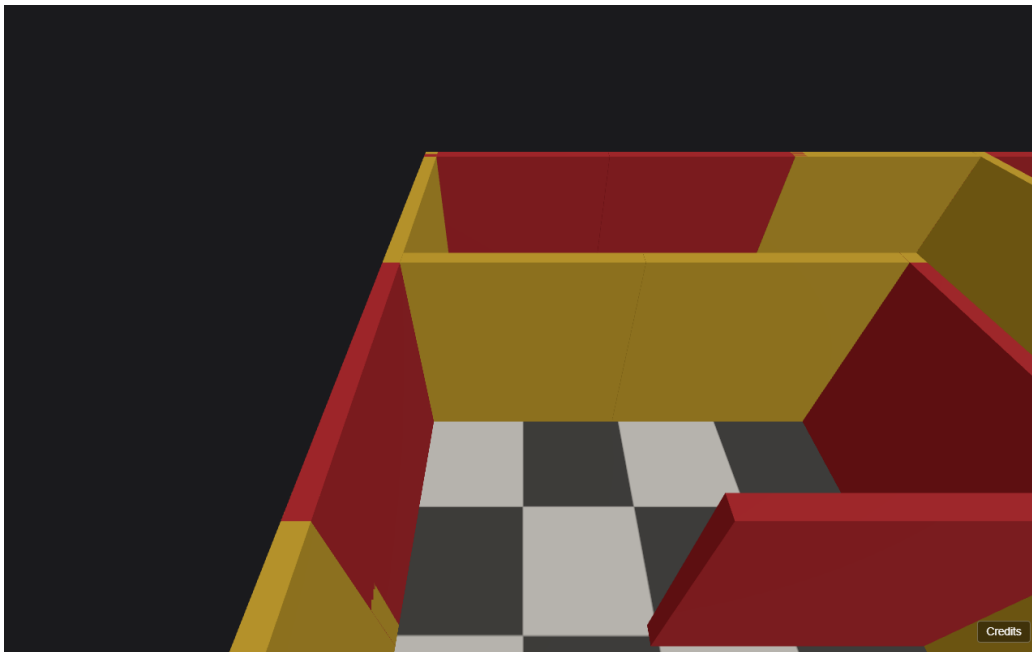


What happens on contact: an immediate retry screen, distinct from reaching the exit.

## Walking through it: collision

Before this level, the game only clamped the player to one big bounding box around the whole room — fine for the two open rooms, but it would have let the player walk straight through every interior wall of a maze. Real wall collision was added specifically for this level:

- Every wall mesh in the maze also produces a 3D bounding box (its collider), built directly from the same geometry that gets rendered — so the walls you see and the walls you collide with can never disagree with each other.
- Each frame, the player is treated as a small circle. If that circle overlaps a wall's box, the player is pushed back out along the shortest direction — simple, cheap, and exact for the axis-aligned walls this maze is built from.



A corridor near the spawn corner, at player eye height. This collision was tested by walking straight into a wall for several seconds — the player stops exactly at its edge with no clipping through.

## Finishing the level

Walking into the exit's trigger volume ends the level immediately: the mouse is released, and a "Maze Complete!" screen appears. Its message changes depending on whether Jax was found first, which is the hook the team can build a proper good/bad ending on top of.



The end-of-level screen, reusing the same overlay the game already shows at startup.

---

## Handing this off

What's already working:

- Maze generation, rendering, and exact wall collision (src/maze/generateMaze.js, src/rooms/mazeRoom.js).
- Jax and exit trigger detection, wired into the existing game loop (src/main.js).
- A "Maze (Level 3)" entry in the room picker, so it's playable right now.
- Jax wandering a short patrol, and a patrolling hazard along the route to the exit — the "navigation + chase" mechanic from the plan.

Still placeholders, left for the team to fill in:

- Swap the purple cylinder for the real Jax model, the hazard for a real abstracted-monster model, and add real exit dressing (a doorway, light, etc).
- Replace the plain text hint/overlay with real dialogue when Jax is found.
- The plan calls for a neon/glitch look for Level 3, distinct from the other two rooms — the current red-yellow-checker look is a placeholder, intentionally left for whoever owns visual effects.
- Only one hazard patrols one segment right now; more hazards or longer patrol routes are a simple extension of the same system.